Debugging Constraint Models with Metamodels and Metaknowledge *

Eugene C. Freuder, Richard J. Wallace and Tomas E. Nordlander

Cork Constraint Computation Centre and Department of Computer Science
University College Cork, Cork, Ireland
email: {e.freuder,r.wallace}@4c.ucc.ie,tomas.nordlander@sintef.no

Abstract. A major challenge in constraint programming is to create an adequate constraint model for a given problem. This is an iterative process, in which an original model must be amended or 'debugged'. The process can often be guided by user knowledge or user preferences, with respect to variable assignments that should appear in some solution. This paper describes a system, "Ananke", which allows this aspect of the modelling process to be partly automated. In Ananke, the current model is considered an instantiation of a metamodel whose solutions are sets of constraints that together form a CSP. An important concept here is that of metaconstraints; these are set constraints based on sets of tuples allowed by an instantiation of the metamodel. If the user suggests partial (or complete) solutions that are not part of the current model, the system uses its metamodel to find changes in constraints that produce a new model with the proposed solutions. This metasearch is guided by metrics for judging quality of revisions and can be aided by heuristics involving optional metaconstraints that improve search efficiency while still guaranteeing the existence of a solution. This work shows how meta-reasoning can be used to support the process of improving constraint satisfaction models as well as discovering new models of potential interest.

1 Introduction.

Modeling combinatorial problems is a multi-faceted endeavour in which finding a feasible (or optimal) solution is often only a small part of the overall task. This has long been recognized in Operations Research, as the following quotation makes clear:

> The discussion thus far has implied that an operations research study seeks to find only *one* solution, which may or may not be required to be optimal. In fact, this usually is not the case. An optimal solution for the original model may be far from ideal for the real problem. Therefore **post-optimality analysis** is a very important part of most operations research studies. [1] p. 22

As this paragraph implies, modeling is an iterative process, and at any stage of this process a model may be incomplete or incorrect. Supporting the overall task will require

* This work was supported by Science Foundation Ireland under Grant 05/IN/1886. We thank Benjamin Jakobus and Séamus ó Buadhacháin for help with the implementation.

an array of problem solving tools as well as the capacity to support extended interactions with the user. Constraint programming, with its toolbox of specialised methods, in the form of constraints, would seem especially in need of (but at the same time well-suited for) extended forms of interaction in which a model is examined, evaluated, and refined.

In this paper, we limit ourselves to a particular form of automated assistance to address a specific class of model deficiencies:

- the user indicates that there should be a set of solutions with a specified property,
- the system indicates that there is not and suggests alterations to the model that will admit such solutions,

and we develop and evaluate procedures for providing this assistance.

This form of assistance can be applied in either of two contexts, which can be termed "debugging" and "discovery". In the first scenario, the user is trying to fix a deficiency in the model. In the second, the user wants to understand the implications of allowing certain combinations of values; here, the intention is to explore the space of models in order to discover new and interesting cases. In most cases, in addition, it is expected that the user is interested in a few assignments where he or she has preferences that are not realized in the current model, rather than full assignments or large partial solutions.

These forms of interaction can occur in a variety of contexts. Consider, for example, a meeting scheduling scenario. There are three meetings to be scheduled, each can be held at 11 or 1, meeting A must be held before meeting B, and meeting B must be held at the same time as meeting C. A specific interaction here might be where the user wants at least one solution where meeting A can be held in the afternoon. Here, the system can suggest relaxing the constraint that meeting A must occur before meeting B in order to allow this solution.

A second example is taken from the domain of university time tabling. Suppose we have gotten a possible schedule, but we would like to have the option of scheduling class X in room Y even though the room is too large. We query the system and it discovers that if a constraint on room size is relaxed, then solutions with this assignment would be possible. A third example concerns job shop scheduling. For example, we might want the option of putting job X before job Y without having to reschedule during the production run. In response to our query, the system finds that if the original precedence constraint $T_x > T_y$, where T_k is the start time of job k, is replaced by a disjunctive constraint that also allows $T_y > T_x$, then this possibility can be accomodated.

An example of a discovery scenario can be taken from the field of configuration. In this case, a change in the model may be desirable because the modeler wishes to shift to a product (solution) that has more options than before and, therefore, relaxes certain constraints. Such cases of "upselling" are discussed in [2].

In all these cases, we would like the system to find reasonable alterations to a model that allow the desired solutions. In addition, the user may want to know

- the effective changes that cause the least amount of alteration to the original model, in some reasonable sense of "least"
- whether effective change can be restricted to some limited part of the model
- what options he or she has in the way of change; in other words, are there alternative kinds of change to choose from?

Along with the latter, the user may want to be able to bias search to favour changes of one sort over others.

This form of interaction, which addresses overconstrained problems, can be viewed as complementary to the interaction in the Matchmaker system [3] where the system suggests solutions to underconstrained problems and the user indicates that these are not solutions and provides alterations to the model to prohibit them. Naturally, the two modes of interaction can be combined. These concerns are also related to work in interactive CSP, dynamic CSP, soft CSP, explanation, and constraint acquisition within the CP community and more broadly to work in knowledge acquisition, truth maintenance, machine learning, and diagnosis.

In particular, the QuickXPlain algorithm [4] for determining preferred relaxations and the CONACQ algorithm [5] for acquiring constraint networks from training sets might be viewed as already addressing specific forms of this general interactive paradigm, and might be usefully extended or combined in this context. However, we wish to explore the possibility that "bespoke" solutions to specific forms of debugging may provide additional efficiency and flexibility. The underlying premise here is that it can be worthwhile to identify specific special cases of the general debugging challenge, and specific methods for dealing with them, then work to generalize, adapt, or combine these methods.

This work is part of a broader research programme to develop a mixed-initiative interface that can provide a variety of assistance modes to address in a user-friendly fashion the variety of knowledge acquisition, validation, and revision issues that can arise in developing and maintaining models for constraint satisfaction and optimisation applications. We have named our project Ananke, after the 'Goddess of Bonds', the deity that rules constraint and coercion.

The next section, 2, introduces metaknowledge and metamodels in the presennt context. Section 3 discusses metaconstraints in this context. Section 4 discusses metrics used during (meta)search. Section 5 discusses heuristics based on metaconstraints. Section 6 presents an overview of the present implementation of Ananke, and describes Ananke's debugging procedure in more detail. Section 7 gives results of preliminary tests of the system. Section 8 discusses related work. Section 9 gives conclusions and a brief note on future prospects.

2 Metaknowledge and Metamodels

We start with the basic model of a CSP, as a triple (X, D, C), where X is a set of n variables, D is a multi-set of domains in 1:1 correspondance with X, and C is a set of constraints applied to X. Also, an assignment is a set of elements each drawn from a separate domain in D; if the assignment is of size n and satisfies all the relations in C, then it is a solution to the CSP model.

2.1 Metaknowledge

We focus our study here by making the following assumptions:

- The property that characterizes the "missing" solutions can be expressed as a set of additional "missing solution constraints" (ms-constraints) over a subset of the problem variables (ms-variables).
- For each constraint in the original problem there is a set of alternative constraints to choose from. In the meeting scheduling example given in the Introduction, we could specify a different form of relation between meetings A and B. Or we could suggest removing the constraint entirely.

This formulation allows for the fact that the user will typically provide only partial solutions involving specific variables or small combinations of variables, often because a complete list of assignments would be impossible to enumerate. It also allows specifications in terms of subsets of domains, and this allows the user to characterize what is missing more abstractly than by simple enumeration.

Our goal here is to assist the user in exploring the space of possible models for the problem under consideration. The user may be saying "I am an expert and I know what combinations of assignments to expect; if this program doesn't provide them it has the wrong model" or the user may be saying "If the model does not allow solutions with these properties, I am willing to change the model".

We will say that the ms-constraints are a form of metaknowledge. In the meeting scheduling example, the ms-constraints would consist of the single constraint that meeting A occurs in the afternoon, and ms-variables the single variable corresponding to meeting A.

2.2 Metaproblems

In Ananke, metaknowledge is used to guide selection of a better CSP model. Search must therefore be conducted in a space of CSP models. In the present formulation, concepts from constraint satisfaction are used to define this new metaproblem. This helps to define the problem in such a way that it becomes a kind of combinatorial optimisation problem that can be solved with depth-first backtrack search.

To this end, we define a constraint metaproblem, or MCSP, as a problem whose solutions are CSPs (viewed as sets of constraints). The values of the metaproblem variables, or metavariables, will be possible constraints. In addition, the MCSP must be augmented to include a representation of the missing solutions. We will call this version of the metaproblem the missing solution CSP, or MS-CSP.

In the meeting scheduling example, the MS-CSP would consist of 5 metavariables, 3 corresponding to the unary constraints specifying the domains of the 3 meeting variables and 2 corresponding to the temporal constraints, and one ms-constraint. The domain of values for the metavariables representing the temporal constraints could be the possible equality/inequality relations between two quantities, or it could be the Allen's algebra constraints. In this example, domains for the metavariables representing the 3 meeting variables each have a *single* unary constraint *value*, specifying the *set* $\{11\ 1\}$ (cf. below). A ground solution to an MCSP is defined to be a solution to one of the CSPs that is itself a solution to the MCSP.

In general, we can have MCSP constraints, or metaconstraints, on the metavariables. In the meeting scheduling example, a same-constraint metaconstraint between

the metavariables corresponding to the temporal constraints could specify that their relations have to be of the same type, e.g both $\neq$.

3 Metaconstraints

Ananke employs metaconstraints to guide the system to changes that respect certain model requirements. The basic metaconstraint is the missing solution constraint. Here, we characterise this more formally, and introduce other metaconstraints. In some cases, particularly the constraints of exclusion, metaconstraints are vital for obtaining new models quickly. In other cases they refine the search by excluding undesirable models.

The major thesis of this section is that in the context of meta-models that do or do not allow a set of assignments, metaconstraints have a generic form. They are all *set constraints*, where the viable sets, those that satisfy the metaconstraint, are sets of tuples that include the requisite set of solutions or partial solutions. This also applies to the metavariables that represent the original CSP variables; in their most general form, their domains are composed of sets, each representing a possible domain of the corresponding variable in the model.

3.1 Representing missing solutions

We will develop our basic ideas by describing missing solution constraints in more depth. A missing solution constraint, C_{ms} can be formalised as a kind of set-constraint whose elements are the possible solution-assignment sets over the set of variables in the solution designated by the user. The possible elements of this constraint, therefore, comprise the powerset of the set of tuples allowed by a set of CSP variables that are (partial) solutions, minus the null set. This can be written:

$$\{\{(X_1/v_1, X_2/v_2 \ldots X_k/v_k)\}\}.$$

X_1, X_2 $\ldots$ X_k are the k variables in the CSP-assignment, where $k \leq n$, the number of variables in the CSP. v_1, v_2 $\ldots$ are assignments to these variables that form a viable (partial) solution. The constraint itself consists of all such solution sets that contain the missing solutions.

For example, suppose a user specifies that variable X_1 should have the value 2, that X_2 should be in the range $1 \ldots 3$, and that X_3 should take the value 5. The tuples consistent with this specification are, (2,1,5), (2,2,5), and (2,3,5). Since any set of these three tuples satisfies the specification, the set of elements satisfying the constraint is the powerset of these three tuples, minus $\emptyset$.

The purpose of this constraint is to constrain the solution set of the CSP. It is therefore a kind of inclusion constraint; that is, it specifies subsets, one of which must be included in the projection of the solution set upon the variables in the desired (partial) solution.

Fortunately, we do not have to represent ms-constraints directly in terms of the formalism given above. Instead, a set of unary constraints is added to each model. These constraints limit the solutions to those that satisfy the ms-constraint. For the example

just given, the constraints would be: $X_1 = 2$, $X_2 \geq 1$, $X2 \leq 3$, $X_3 = 5$. For the meeting-scheduling example, the added constraint would be $A = 1$. In this way we search for CSPs that satisfy the ms-constraint as well as the other metavariables (and metaconstraints: see below). If search is successful, removal of these unary constraints gives a CSP with the desired solutions.

The argument for implementing ms-constraints in this way is straightforward. We know that the set of solutions of the original CSP does *not* satisfy the ms-constraint. However, any selection of values for the metavariables consistent with the tuple-sets specified by the user will be included in the set of solution-sets that satisfies the ms-constraint. This is because the set of partial assignments is a subset of the intersection of all solution sets that contain this set as a subset, i.e.

$$\{(specif.\ assign.)\} \subseteq \bigcap \{\{solns\ incl.\ specif.\ assigns.\}\}$$

If, in addition, we choose values for the metaconstraint variables that serve to relax the original CSP constraints, we will not lose any of the original solutions. The present implementation is organised to make selections according to this requirement.

3.2 Constraints of exclusion

Constraints of exclusion restrict search to specific parts of a model. At present, Ananke includes two major types, in the form of (i) a system for prioritising constraints, which includes specifying hard constraints (this means that the present constraint is the only acceptable one in the domain), (ii) specifying a focus in the problem graph by means of "focal constraints".

Priorites are a form of unary metaconstraints. Focal constraints are metaconstraints whose scopes do *not* include some of the metavariables. They specify that the metavariables within their scope must take the values represented by the corresponding constraints in the original CSP model. Their purpose is to exclude these metavariables from the metasearch. This is especially important with regard to metavariables associated with the ms-variables, as shown in the Heuristics section below.

As noted above, constraints based on priorities and focal constraints are both types of set constraints. However, given a search criterion of minimal change in the original model and an assumption of monotonicity with respect to domain values (i.e. constraints with the same scope), we do not have to represent these sets explicitly. Obviously, all we need to do is exclude constraints associated with priorities of 1 and metavariable subject to focal constraints from consideration during meta-search.

3.3 Constraints of order

Constraints of order create dependencies among the changes to the model. Because they associate different parts of the model, we call them "linking constraints". Such constraints can include simple equalities (as in the example given above) and more general "homogeneity" constraints that restrict the degree of variation in changes to different parts of the model (similar to the meta-constraints described in [6]).

As with focal constraints, in many cases we can avoid direct representation of sets of tuples. This is because, given a monotonic orderings of (CSP-)constraint-relaxations, these constraints can usually be evaluated by simple procedures, e.g. are the present relaxations within k steps of each other?

4 Metrics for Meta-Search

Finding ground solutions to the MS-CSP involves choosing metavariable values, i.e. constraints, and testing for ground solutions. We can make these choices with some form of backtrack search, as described below. This raises questions of efficiency, but also of the quality of solutions, since there are many alternative ways that quality could be measured. Both issues involve metrics on which CSP models can be compared.

A natural consideration in judging quality of suggestions is the amount of change from the original model entailed by a suggestion. Here, we make the assumption that changes in the model can be ordered in terms of the degree of change, and that small changes are preferred to large ones.

Perhaps the most fundamental metric of this kind is the number of additional solutions allowed by the altered model. But since computing all solutions is expensive, we will consider cruder metrics that are correlated with this one.

For constraints based on relational operators ($<$, $\leq$, $\neq$, etc.), a simple metric is obtained by arranging the possible constraints in a partial order, in which any change that loosens a constraint, i.e. allows more tuples to satisfy it, is judged to have greater cost than a tighter constraint. For example, single steps in the resulting lattice can be considered to add a cost of 1 to the overall cost of the change.

A more sophisticated metric that is still often simple to compute is based on the number of tuples that would be added if the constraint were relaxed in a certain way. For example, if a binary $>$ constraint were relaxed to $\geq$, this would add d tuples (given two domains of size d), while if it were relaxed to $\neq$, then $d^2 - d$ tuples would be added. This strategy allows us to derive a universal metric for relaxing constraints, that can be represented as a lattice whose elements are associated with a number of tuples ranging from 0 to d^k, where k is the arity of the constraint. It is universal in the sense that whatever the set of relaxations allowed, the values of the metric will range between these two values.

When a metric is added to the metavariables in a MS-CSP representation, the result is a kind of soft constraint system [7]. However, the kind of soft CSP or even the kind of formalism that is most appropriate can vary with the features of the metric. If the step-based metric just described is used, the MS-CSP together with this metric is a kind of valued CSP. On the other hand, the tuple-based metric requires the kind of representation given by the semiring formalism. For example, it is consistent with a semiring in which tuples in the original constraint are given a weight of 0 while tuples allowed by looser constraints are each given a weight of 1.

Because there is no single soft constraint formalism that is most appropriate under all circumstances, and because the metric can be uncoupled from the basic MS-CSP concepts presented in the previous section, the metaCSP framework can be viewed as a generic description apart from the soft CSP representation. In fact, a MS-CSP does not

become a soft constraint system until a metric is introduced. (At the same time, it should be noted that any metric including step metrics and mixtures of step and tuple metrics applied to different constraints can be represented as semirings.) Moreover, as noted by [6], soft constraint systems are not well-suited for expressing metaconstraints. Finally, we should note that the present system is distinctive in that cost metrics are related to the number of solutions allowed by a model, unlike the usual soft constraint system where constraint or tuple costs reflect evaluations of particular states of the domain represented by the model.

The same step/tuple dichotomy obtains for constraints such as AllDifferent. Here, a step-form of revision is to reduce the number of variables in the scope of the constraint that are required to be different (i.e. instead of all k variables being different the requirement is for $k-1$ of the k variables to be different, etc.). A tuple-form of revision is to require that the extent of difference across assignments to variables in the constraint be minimised; relaxations of this restriction are then associated with higher costs.

As with other soft constraint systems, metric values for different constraints can be combined in a number of ways. In the present work, we have simply rescaled different metrics so their upper limits are the same, and used a weighted sum of scaled values to obtain a metric, or cost function, for comparing CSPs. Here, weights represent (negative) preferences among the constraints, reflecting differences in willingness to relax a given constraint. Constraint weights can also be adjusted to reflect user priorities, e.g. that some constraints should be retained in their present form, others should only be relaxed if necessary, etc.

5 Heuristics for Meta-Search

To make meta-search more efficient, we can use constraints of exclusion, in particular focal constraints. The basic strategy is to determine which of the variables in the ms-constraint can be assigned a value consistent with the expected (ground) solutions, and then to focus the meta-search on variables that could not be assigned. In the extreme case, we use focal constraints to exclude constraints that are not associated with metavariables corresponding to the unassigned variables in the original model. As a result, meta-search involves only metavariables that represent constraints whose scopes include these unassigned variables.

If the metasearch is restricted by focal constraints, we refer to this as *focused search*. In this context, the following proposition is obvious but essential:

Proposition. *If focused search is limited to metavariables that represent constraints adjacent to unassigned variables, then a solution can always be found.*

Proof. This follows from the fact that if the original constraints on these variables are removed (i.e. the values of the metavariables are those that allow all possible tuples), then the resulting model allows the variables in question to be assigned the expected values. Since all the other variables could be assigned in the original model, the resulting model allows the expected solutions. □

Although removing a constraint in the focus will always produce a solution, it may not be as desirable as the best-cost solution obtained by searching through a larger space of possible changes. On the other hand, if one of the criteria for goodness of a solution is that the changes occur as close to the failed assignment as possible, then focused search may give a preferred solution. In addition, the proposition can be generalised to sets of variables that include the unassigned variables.

Given this form of heuristic search, several approaches could be used to determine the focus:

- taking constraints adjacent to the unassigned variables in a best-match solution (i.e. a solution that comes closest to one that allows the user's expected solution or assignments),
- taking the union of the unassigned variables in all best-match solutions,
- allowing the user to choose from a selection of best-match solutions.

The most straightforward interpretation of "best-match" is a maximal match, and this is used in the present version of Ananke. In addition, the focus can be enlarged by taking constraints in some neighborhood of the variables in the focus; this extension can be combined with any of the above approaches.

If the user opts for metaconstraints of this form, this introduces an additional search problem, since the location of the focus depends on finding the variables not included in a maximal match between solutions in the present model and the set of expected solutions. The most straightforward way to do this search is to find all solutions of the original CSP model, keeping track of solutions that match the desired solutions in as many assignments as possible. However, efficiency considerations rule out this approach for anything other than small CSP models with a few solutions.

The algorithm used in the present version of the system begins with the full expected solution (k sets of assignments), added to the model as a set of unary constraints. If there is no (CSP) solution, then the model is tested with each of the k sets of $k-1$ assignment sets, then with the $k(k-1)/2$ sets of $k-2$ assignment sets, etc. (Successive sets of variables to assign can be found in $O(n)$ time by indexing the variables and representing the selections as array entries.) Under the maximal match criterion, search can stop when one or all maximal matches have been tested.

The worst-case complexity for the full search is bounded by the expression,

$$\sum_i \binom{m}{i} S_{n-m+i}$$

where n is the size of the CSP, m is the size of the missing solution, and S stands for search-effort, itself bounded by $d^{n-m+i}d'^{m-i-s}$. (s represents portions of the specified solutions that are single assignments.) Nonetheless, in practice this method is often strikingly more efficient than an all-solutions search on problems of small to moderate size (see below). This means that the system can find all maximal matches, which often gives the user considerable flexibility in specifying the location of the focus.

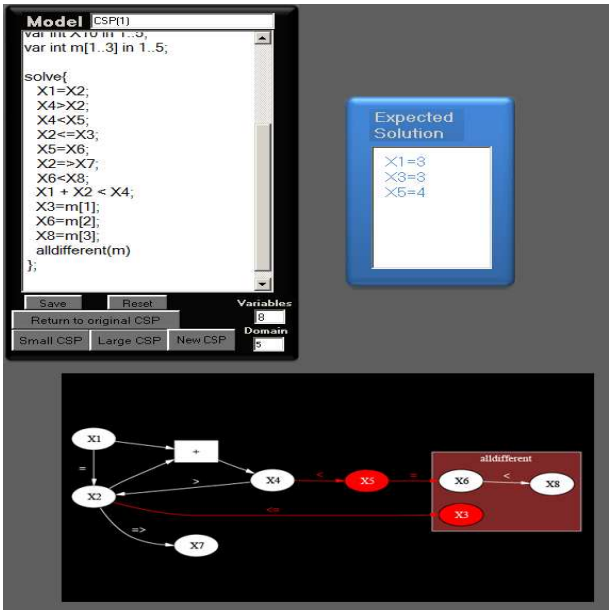

Fig. 1. Three panels from Ananke Interface showing model, expected solution, and constraint graph. In the graph, the two shaded nodes are in the focus, which also includes four constraints.

6 Interface and Operation

In addition to developing a platform as a concrete realisation of the ideas presented above, an important aim of our system is to allow these features to be used as flexibly as possible. By including a number of options, the system allows the user to specify the way in which the metrics and metaconstraints should be employed.

In the present version of Ananke, the user has options with respect to:

- the elements the missing solution constraint (varied by altering the maximal match),
- the scope of the focus (which elaborates the option related to the ms-constraint)
- the form of the metric, in particular, whether a step- or tuple-based metric is to be used to find a minimal relaxation of the original model.

The first two options are supported by collecting a set of maximal matches. The user is allowed to browse the set of maximal matches (specifically assigned and unassigned variables), to see which parts of the problem contain the uninstantiated variables and to evaluate the appropriateness of different sets of focal constraints. During browsing, the user is informed of the number of solutions associated with each maximal match (up to a limit set by the user). The assumption here is that the user may prefer a maximal match associated with more solutions, because in this case it is more likely that a viable model can be found that is closer to the original model.

In addition, the user can enlarge the focus beyond the original focal constraints. This is done by stepwise inclusion of neighbors of the original uninstantiated variables,

the neighbors of these neighbors, etc. The user can futher control the set of constraints considered in the focus by setting priorities; for example, making a constraint hard takes it out of the focus entirely.

A third goal of the implementation is to support experimentats on model debugging. To this end, the user can choose the types of constraints to include, and the kinds of adjustments that can be made during search. For example, with relop constraints change can be restricted to quantitative variation or to steps in the lattice of relational operators. In addition, basic problem parameters such as number of variables, domain size and constraint graph density can be specified, as well as the size of the desired partial solution.

The present version of the system is implemented in Visual Studio; the code is written in Visual Basic. The system runs under Windows XP on a Dell Precision PWS 390 (2.66 GHz, 2 GB RAM). ILOG solver is used to solve specific CSP models. The present system supports constraints based on relational operators ($>$, $\geq$, =, $\neq$, $<$, $\leq$). This includes both binary constraints and $n$ary constraints of the form

$$X1 + X2 + ... + Xk \otimes Z$$

where $\otimes$ is a relational operator.

In addition, allDifferent constraints are allowed. At present, the type of relaxation is limited to a single $k - r$ allDifferent constraint, where k is the original constraint arity and r is the cardinality of the set of variables excluded from the original constraint, which can vary from 1 to $k - 2$. Moreover, because the number of possible relaxations becomes large as k grows, at present the metasearch can only be performed with allDifferent constraints of low arity.

Portions of the basic user interface are shown in Figure 1. These are a panel for showing the current model in text format, a panel for showing the desired solutions, and a panel for displaying the constraint graph. In addition, a control panel allows setting experimental parameters such as problem size, size of expected solution, etc. and another panel shows run statistics. In the constraint graph, variables that could not be matched for a given maximal-matching solution are indicated by shading.

In the present version of Ananke, debugging meta-search is done with a branch-and-bound algorithm that uses a specified cost-function to find minimal-cost changes to the present set of constraints (Figure 2). The solutions sought in this process are sets of constraint changes that give the best cost according to some metric. The number of constraints in such a set can vary from 1 to c. However, if we allow the null change as one of the possible changes in a constraint, then all constraints will be involved in every solution; obviously, we can obtain the solution we want by discarding those constraints with null changes after search is over. This allows us to treat the metaproblem as an ordinary CSP whose variables are the constraints potentially subject to change, and whose values are the allowable changes.

During depth-first search, the algorithm employs a value-ordering heuristic that orders changes in a given constraint according to the cost function (metric) employed. In many cases, changes can be preordered even if the exact cost is not known ahead of time. For example, a $\geq$ constraint will always allow at least as many tuples as a $>$ constraint. This simple heuristic greatly enhances the bounding process (in the present

monotonic context), because once the current bound has been exceeded for a given constraint change, then any change given by an alternate value for this metavariable will also exceed the bound.

```
meta-branch-and-bound()
currentCost = 0
bestCost = ∞
searchLevel = 0

while searchLevel ≥ 0
    if searchLevel > n              'new solution found
        create and save newCSP based on present metamodel
            and ms-constraints
        if newCSP has a (CSP) solution
            if currentCost < bestCost
                set bestSolutions to solution
                set bestCost to currentCost
            else
                add solution to set of bestSolutions
        decrement searchLevel

    else if nextRelaxation ≤ maxRelaxation at searchLevel
        if currentCost + cost of nextRelaxation > bestCost
            reset data structures at this level
            decrement searchLevel
        else
            add nextRelaxation to partialSolution
            save nextRelaxation and update currentCost
            increment searchLevel
    else
        reset data structures at searchLevel
        decrement searchLevel
```

Fig. 2. Pseudocode for branch and bound algorithm for search for the set of best-cost solutions in a space of metamodels using a monotonic cost function.

7 Evaluation of System Performance and Search Strategies

Tests discussed in this section were performed on problems where all constraints were binary of the form $Xi \otimes Xj$, where $\otimes$ is a relational operator. For problems of this sort, we are able to debug models of moderate size (up to about 25 variables depending on the problem parameters and the metrics and metaconstraints), although the size depends strongly on whether search is focused or unfocused and on the size of the maximal match. Since the present limitation is due to network connections with the ILOG solver, we expect that the basic system in its present form will scale beyond this limit.

The first tests we did were intended to determine the efficiency of the iterative maximal matching algorithm in comparison with all-solutions search. In experiments on problems with 15-20 variables, small domains (5 values) and low to medium density (0.10 – 0.25), iterative maximal matching algorithm outperformed the all-solutions search by three orders of magnitude (0.015 vs. 30-40 seconds). This time includes the solution-counting feature with a 1,000 solution limit. Hence, with this algorithm and for the problems we are currently using, obtaining a set of maximal matches is not an important bottleneck.

We also ran experiments to determine the quality and numbers of suggestions under (strict) focused and unfocused search. In these experiments, 25 problems were generated per condition.

We describe a typical experiment. The number of variables was ten, domain size was five, density was 0.2. Constraints were all binary relop. The size of the set of desired assignments was set at 7. For further experimental control, debugging metasearch was restricted to cases where the initial search found a maximal match of 0.71 (2 uninstantiated variables). The results are shown in Table 1. In this as in other experiments, unfocused search did find more suggestions, and with a more refined metric, the cost per suggestion was reduced.

Table 1. Results for Focused and Unfocused search

	#suggestions		chgs per sugg		cost per sugg	
metric	focused	unfocused	focused	unfocused	focused	unfocused
step-based	1.25	1.65	2.2	2.4	2.2	2.4
tuple-based	1.0	1.3	2.8	2.1	23.0	17.0

8 Related Work

The present research is related to work on constraint acquisition. This work involves procedures derived from psychology and machine learnig for acquiring discrete concepts based on examples; in this case, the examples are solutions and non-solutions. In this context, much effort has been put into developing efficient sets of queries that will enable a CSP model to be obtained [8] [5]. However, since these queries involve complete solutions, it is not clear how well they will scale or whether this approach will be useful to users if they have to evaluate full solutions.

In the present work, we are attempting to adjust an existing model. Thus, while work in constraint acquisition attempts to impose restrictions on a space of possible models, the intention of the present work is to find models that are minimally distant from an existing model. As a result, the techniques employed in these respective tasks are quite different from those used so far in constraint acquisition. Nonethess, it may be possible to combine the present approach with concept learning, where the system not only finds a set of minimally distant solutions but then presents examples in order to distinguish among them (or even larger sets).

In the present debugging task, the distinction between complete and partial solutions is not critical, although our algorithms are naturally more efficient when the solution set is more restricted. To our knowledge this has not been considered in the constraint

acquisition context. Also, we should note that when the set of possible constraint relaxations is extended, e.g. if X $>$ Y can be replaced by X $>$ Y - k instead of by another relational operator, this will not have a marked effect on task difficulty although the version space is greatly extended with consequent difficulties for pure acquisition.

Another related area concerns "explanations" for assignment failures (e.g. [9] [4] [10] [11]). This work considers a similar type of problem, but from the other 'end of the telescope', in that the goal is to find adjustments to an invalid complete or partial solution that will satisfy the constraints that disallow it. In other words, the emphasis in explanation research is on finding correct solutions, while our concern is properly defining the problem. For this reason, it is not necessary to move to the level of meta-knowledge to obtain explanations. It may be possible to use minimal conflict sets as heuristics for guiding the debugging process. On the other hand, since conflict sets of this sort involve variables in the expected solution, they are not always associated with minimal relaxations of the model, which may occur in parts of the constraint that are distant from the variables in the expected solution.

The context established here of searching for alternative models in a space of possibilities with a distance metric is reminiscent of the partial CSP (PCSP) framework for overconstrained problem solving [12]. In fact, in that earlier work a search through a space of relaxed CSPs was considered but that approach was not pursued. Since the concern in the PCSP context was to find an existing assignment with optimal properties (e.g. minimal number of violated constraints), the algorithms were different in character from the one presented here. Again, however, there is a potential for interplay between that field and the present work.

Constraints-of-order metaconstraints are related to the general concept of meta-constraints introduced in [6]. Like those authors, we wish to use metaconstraints to "express global rules about violated constraints" (*ibid*, p. 359). For example, they describe meta-constraints that add homogeneity criteria with respect to soft constraint violations as well as dependencies among violations. However, metaconstraints in this form must be distinguished from the specialised forms of constraint used in conjunction with auxilliary variables (called "distance" and "disjunctive constraints"), that are also referred to as meta-constraints by these authors, and are the eventual focus of their paper. This difference stems from their concern with over-constrained problems that have no solutions. Hence, their goal is to find the best partial solutions to a given model rather than a new model that can accomodate specific (partial) solutions.

9 Conclusions and Future Work

In this paper we have presented a new approach to constraint model debugging, and we have demonstrated the viability of these ideas with a working implementation. There are many extensions to consider. Obviously, the class of constraints and relaxations can be extended further. More importantly, we need to consider the source(s) of meta-knowledge in greater depth. That is, how do we decide what the alternatives for a given constraint should be? There is also much to be done to encode knowledge about problem classes, such as scheduling or time tabling problems. It should be possible to do this within our system of metrics and metaconstraints.

Although the present system only makes suggestions that are monotonic with respect to allowed solutions, we may also want to operate in a non-monotonic context. The changes we wish to make may remove some solutions at the same time as they add others. Since the system is assisting the user in searching the space of possible models, it needs to help the user avoid 'infinite loops'. On the other hand, users may want the right to change their minds, and e.g. remove solution s to admit solution t. Ultimately, we would like to be able to provide the user with a choice as to how 'paternal' the system should be, e.g. the user might say "never make a suggestions that would cause the removal of a solution I have added" or "only make such a suggestion as a last resort" or "tell me whenever you are making such a suggestion".

We believe that using metaknowledge supplied by human-computer interaction to guide search through a metaspace of models provides a powerful approach to the CSP acquisition challenge. The broader vision for the future of Ananke is an environment that can provide constraint knowledge acquisition and maintenance assistance in a variety of forms to take best advantage of the user's experience while moving as much of the burden of the process as possible from human to machine.

References

1. Hillier, F.S., Lieberman, G.J.: Introduction to Operations Research. 5 edn. McGraw-Hill (1990)
2. Hulubei, T., Freuder, E.C., Wallace, R.J.: The goldilocks problem. Artificial Intelligence for Engineering Design, Analysis and Manufacturing **17** (2003) 3–11
3. Freuder, E.C., Wallace, R.J.: Suggestion strategies for constraint-based matchmaker agents. International Journal on Artificial Intelligence Tools **11**(1) (2002) 3–18
4. Junker, U.: Quickxplain: Conflict detection for arbitrary constraint propagation algorithms. In: IJCAI 2001 Workshop on Modelling and Solving Problems with Constraints. (2001)
5. Bessière, C., Coletta, R., O'Sullivan, B., Paulin, M.: Query-driven constraint acquisition. In: Proc. Twentieth Internatioal Joint Conference on Artificial Intelligence-IJCAI'07. (2007) 50–55
6. Petit, T., Régin, J.C., Bessière, C.: Meta-constraints on violations for over constrained problems. In: Proc. Twelfth International Conference on Tools with Artificial Intelligence-ICTAI'00. (2000) 358–365
7. Bistarelli, S., Montanari, U., Rossi, F., Schiex, T., Verfaillie, G., Fargier, H.: Semiring-based CSPs and valued CSPs: Frameworks, properties and comparison. Constraints **4** (1999) 199–240
8. O'Connell, S., O'Sullivan, B., Freuder, E.C.: A study of query generation strategies for interactive constraint acquisition. In: Applications and Science in Soft Computing. (2003) 225–232
9. Jussien, N., Barichard, V.: The paLM system: Explanation-based constraint programming. In: Proc. CP 2000 TRICS Workshop. (2000) 118–133
10. Amilhastre, J., Fargier, H., Marquis, P.: Consistency restoration and explanations in dynamic csps - application to configuration. Artificial Intelligence **135** (2002) 199–234
11. O'Callaghan, B., O'Sullivan, B., Freuder, E.C.: Generating corrective explanations for interactive constraint satisfaction. In: Principles and Practice of Constraint Programming - CP2005. LNCS. No. 3709. (2005) 445–459
12. Freuder, E.C., Wallace, R.J.: Partial constraint satisfaction. Artificial Intelligence **58** (1992) 21–70